W5 - PRACTICE

Props – List display

At the end of this practice, you should be able to…

- ✓ Display **names dynamically**, using **curly braces** *{xx}*
- ✓ Display a **list** of data
- ✓ Display **images** dynamically
- ✓ Pass data to nested components using **props**

How to work?

- ✓ Download **the start code** from the Google classroom
- ✓ For each exercise you can either:
  - Run `npm install`
  - Or move an existing `node_modules` to the exercise folder *(fastest option!)*

How to submit?

- ✓ **Create a repository on GitHub** with the name of this practice:
  Ex: C2-S1-PRACTICE
- ✓ **Push your final code** on this GitHub repository (if you are lost, follow this tutorial)
- ✓ Finally, submit on **Google classroom** your GitHub repository URL
  Ex: `https://github.com/thebest/` C2-S1-PRACTICE`.git`

Are you lost?

You can read the following documentation to be ready for this practice:

https://www.w3schools.com/react/react_jsx.asp
https://www.w3schools.com/react/react_props.asp
https://www.gatsbyjs.com/docs/how-to/images-and-media/importing-assets-into-files/
https://www.w3schools.com/react/react_conditional_rendering.asp

LAB5.1: 4-Level User Profile Drill

Task: Pass the `user` object from the top to the bottom without using any state in the middle components. **Startup Code:**

```
const AuthorInfo = ({ user }) => { /* Display user.name and user.role here */ };
const PostHeader = ({ user }) => <AuthorInfo user={user} />;
const PostItem = ({ user }) => <PostHeader user={user} />;
const FeedList = ({ user }) => <PostItem user={user} />;

function App() {
  const [user] = useState({ name: "Ronan", role: "Admin" });
  return <FeedList user={user} />;
}
```

LAB5.2: Layout Wrapper (Children Prop)

Task: Use the `{children}` prop to create a reusable shell for the app

```
const Layout = ({ children }) => {
  return (
    <div className="app-container">
      <nav>Navbar</nav>
      {/* Task: Place children here */}
      <footer>Footer</footer>
    </div>
  );
};
```

LAB5.3: Asset Loading (Image State)

Task: Handle the case where a post has no image.

```
const PostImage = ({ url }) => {
  // Task: If url is null, show a gray <div>. If url exists, show <img>.
  return <div className="img-wrapper"></div>;
};
```

LAB5.4: Multiple Callback Actions

Task: Pass two different functions down to a child component to handle two different buttons.

```
const PostActions = ({ onLike, onShare }) => {
  return (
    <div>
      <button onClick={onLike}>Like</button>
      <button onClick={onShare}>Share</button>
    </div>
  );
};
```

LAB5.5: Identify the Child (ID Callback)

Task: Pass an ID back up to the parent when a button is clicked.

```
const DeleteBtn = ({ id, onConfirm }) => {
  return <button onClick={() => { /* Task: Call onConfirm with ID */ }}>Delete</button>;
};
```

LAB5.6: CREATE (Object State)

Task: Take input values and add a complete object to the array state

```
function App() {
  const [posts, setPosts] = useState([]);
  const [newContent, setNewContent] = useState("");

  const handleAdd = () => {
    // Task: Update posts with a new object { id, text: newContent }
  };
}
```

LAB5.7: READ (Filtering Data)

Task: Create a search bar that filters the list being displayed.

```
function App() {
  const [search, setSearch] = useState("");
  const items = ["React", "JavaScript", "Tailwind"];

  // Task: Create filteredItems variable before mapping
}
```

LAB5.8: UPDATE (The "Edit" Toggle)

Task: Find an item in the array by ID and change one property (e.g., `isEdited`)

```
function App() {
  const [posts, setPosts] = useState([{ id: 1, text: "Hello", isEdited: false }]);

  const markAsEdited = (id) => {
    // Task: Map through posts and flip isEdited for the matching ID
  };
}
```

LAB5.9: DELETE (Filter Method)

Task: Remove an item from the UI using the `.filter()` method.

```
function App() {
  const [list, setList] = useState([{ id: 1 }, { id: 2 }, { id: 3 }]);

  const removeItem = (id) => {
    // Task: setList using filter
  };
}
```

LAB5.10 *(Start code - EXERCISE 1)*

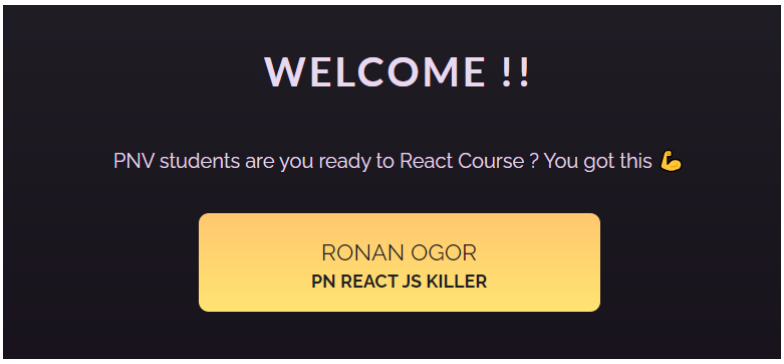
Your task is to fill the **User** component!

You have a global existing userData, containing a first name, last name, and title.

- Output the **user's first name** and **last name** () inside of the User component's <h2> element.
- Output the **title** inside of the User component's <p> element.

Note: you can change the user data 😊

The finished app could look like this:



LAB5.11 *(Start code - EXERCISE 2)*

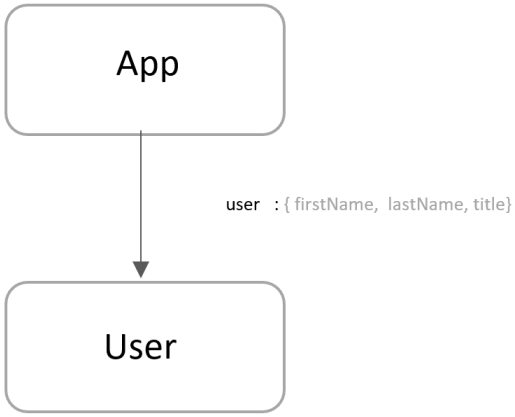
Well done!

But now we have 2 people to display using this **User** Component: Vinh and My Ngo!

Note: you CANNOT create 2 components, you need to use the user component and pass data to this component

The finished app could look like this:

App will provide a user data to each User component:



LAB5.12 *(Start code - EXERCISE 3)*

Amazing!

But now we need to display a bigger list of PNV teachers, using this **User** Component…

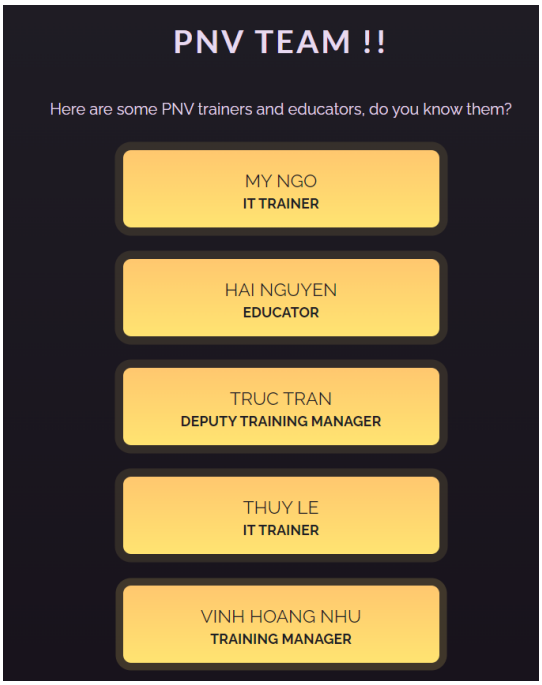
The data comes from `teachers.js` and we already imported this list.

```
export const ALL_PNV_TEACHERS = [
  {
    firstName: "My",
    lastName: "Ngo",
    title: "IT Trainer",
  },
  {
    firstName: "hai",
    lastName: "Nguyen",
    title: "Educator",
```

All you need is to loop on this list now and **create a User component for each teacher.**

Need help? You can [read this doc](#) to render lists with a loop

The finished app could look like this:



LAB5.13 *(Start code - EXERCISE 4)*

Amazone!

Now we want to display a **list of places.**

- The places come from `data.js` and we already imported this list.
- Each place shall be displayed using the **Place** component

The mapping between Place component attribute and the data is the following one:

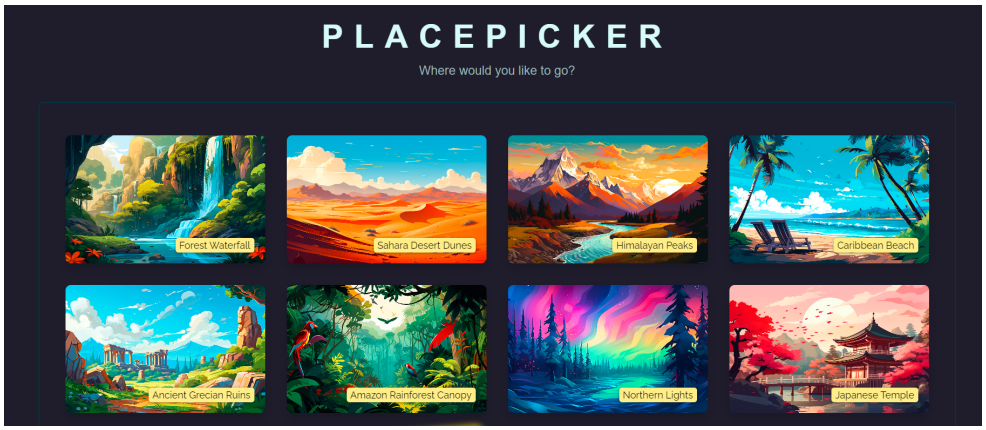
Place component

```
<li key="the place id" className="place-item">
     <button>
       <img src="the place image src" alt="the place image alt" />
       <h3>"the place title"</h3>
     </button>
</li>
```

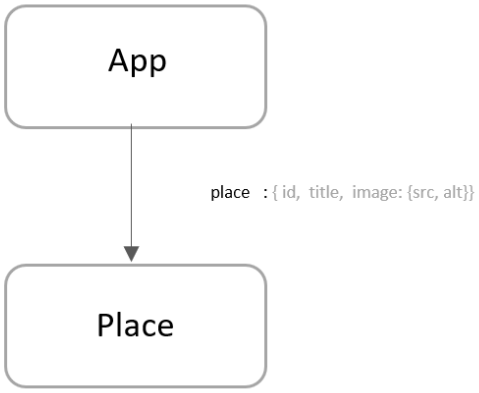
Place data

```
{
    id: "p1",
    title: "Forest Waterfall",
    image: {
      src: forestWaterfallImg,
      alt: "A tranquil forest with a cascading waterfall",
},
```

The finished app could look like this



App will provide a place data to each Place component:



LAB5.14 (Start code - EXERCISE 5)

Amazoooooooooooone!

For this last exercise, you need to show your **creativity**!

1. You need to start from an **empty app.**
2. You are **free** to choose your design and content.

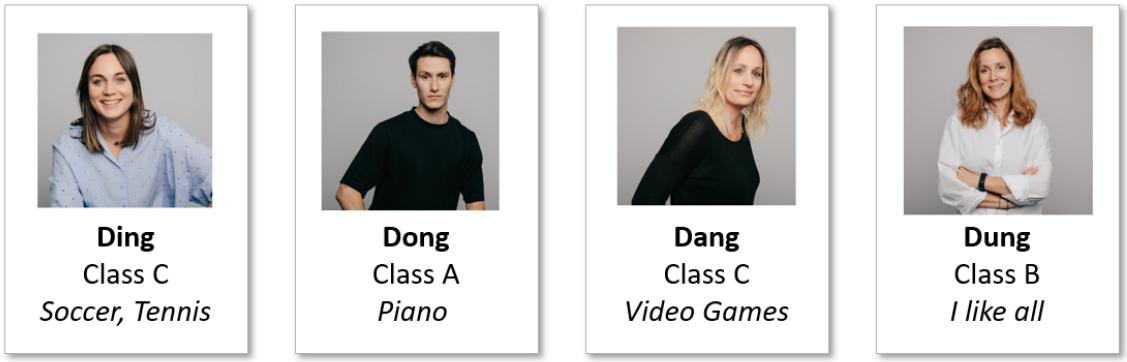
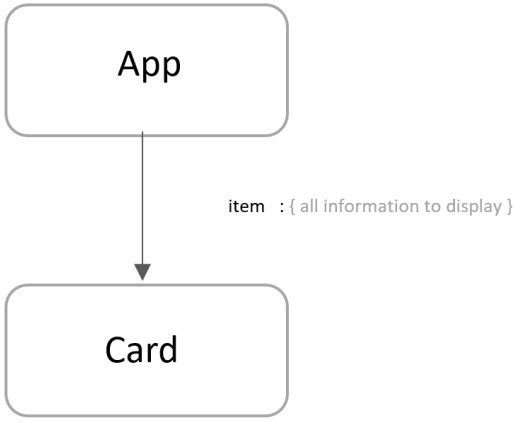
3. Your app shall display a list of cards using a **Card** component. Each card shall contain:
   - A picture of someone or something
   - Extra information (for example: *name, hobbies, gender, address, price…)*

4. All data shall be stored in the file: **`data.js`**

Are you lost in CSS and HTML?

We have provided an example of HTML/CSS code in the following folder: **html-sample-code**

Example



App will use a Card component to display the items: